

Nombre del Proyecto	Sistema para monitorear el nivel de agua de un tanque utilizando ESP32.
Equipo de trabajo	Estefannia Benjumea, Víctor Gómez, Johan Andrés Isaza.
Grupo	1
Docente	Erick E. Reyes Vera
Repositorio GitHub	[URL del repositorio]
Video demostrativo	[URL del video / carpeta institucional con respaldo]

TABLA DE CONTENIDO

1. Información general del proyecto.....	1
2. Resumen ejecutivo.....	2
3. Planteamiento del problema y contexto ABP	2
4. Objetivos general y alcance del proyecto.....	3
Objetivo general.....	3
Objetivos específicos	3
Alcance del proyecto.....	3
5. Estado del arte	3
6. Materiales, herramientas y consideraciones de seguridad	4
Hardware	4
Software.....	5
Herramientas adicionales	5
Consideraciones de seguridad	5
7. Diseño electrónico, simulación y montaje	6
8. Algoritmo o código empleado	7
9. Integración con nube, dashboard y visualización	14
10. Protocolo de pruebas, resultados y análisis	15
Descripción de la prueba realizada	15
Descripción de la gráfica.....	16
11. Video demostrativo y evidencias audiovisuales	16
12. Impacto ético, social, ambiental y de sostenibilidad	17
13. Conclusiones y aprendizajes	17
14. Referencias y anexos	18

1. Información general del proyecto

Título del proyecto:

Sistema para monitorear el nivel de agua de un tanque utilizando ESP32.

Problema que atiende el proyecto:

En algunos lugares donde se almacena agua en tanques, no siempre es fácil saber cuánta agua hay disponible. Esto puede ocasionar desperdicio cuando el tanque se llena demasiado o dificultades para controlar el consumo. Por esta razón, surge la necesidad de contar con una herramienta que permita conocer el nivel del agua de forma rápida y sencilla.

Tema seleccionado del banco de proyectos:

Monitoreo inteligente de nivel de agua.

Nombre del prototipo:

Monitor Inteligente de Nivel de Agua.

Herramientas y componentes utilizados:

- ESP32
- Sensor ultrasónico HC-SR04
- Pantalla OLED
- LEDs
- Buzzer
- MicroPython
- Wokwi

Repositorio GitHub:

Pendiente de agregar enlace.

Video demostrativo:

Pendiente de agregar enlace.

Fecha de entrega:

2 Junio de 2026.

Integrante	Rol asignado	Responsabilidades principales
Estefannia Benjumea Osorio	Líder de integración y documentación	Coordina la organización del proyecto, realiza el informe, reúne las evidencias y verifica que toda la documentación esté completa para la entrega final.
Víctor Gomez	Responsable de hardware y simulación	Diseña el circuito en Wokwi, realiza las conexiones de los componentes, verifica el funcionamiento de la simulación y apoya la validación del montaje.
Johan Andres Isaza	Responsable de programación y pruebas	Desarrolla y ajusta el código del ESP32, realiza las pruebas de funcionamiento, analiza los resultados obtenidos y apoya la presentación del proyecto.

2. Resumen ejecutivo

Se desarrolló un sistema inteligente para monitorear el nivel de agua de un tanque, con el objetivo de evitar problemas como desperdicio de agua o quedarse sin suministro por no conocer el nivel disponible. La solución consiste en un sistema IoT que utiliza un sensor ultrasónico HC-SR04 para medir la distancia entre el sensor y el agua, mientras un ESP32 procesa la información y muestra el porcentaje de llenado en una pantalla OLED. Además, el sistema enciende luces LED y activa un buzzer como alerta cuando el nivel del agua está bajo, medio o alto.

Para el desarrollo del prototipo se utilizaron un microcontrolador ESP32, un sensor ultrasónico HC-SR04, pantalla OLED, LEDs y buzzer, programados en MicroPython y simulados en la plataforma Wokwi.

Como resultado principal, el sistema logró medir el nivel del agua en tiempo real y mostrar la información de manera clara, además de generar alertas automáticas durante las pruebas realizadas. Esto permitió demostrar el funcionamiento básico de un sistema IoT aplicado al control de nivel de líquidos. La principal limitación fue que el proyecto solo se probó en simulación y maqueta, por lo que como mejora futura se propone implementarlo en un tanque real y agregar monitoreo remoto desde internet.

3. Planteamiento del problema y contexto ABP

En la ciudad de Medellín el servicio de agua potable suele ser constante; sin embargo, en algunos conjuntos residenciales, instituciones, fincas o sistemas de reserva se utilizan tanques para almacenar agua. En muchos casos, el nivel del tanque se revisa de manera manual o simplemente por observación, lo que puede ocasionar desperdicio de agua cuando el tanque se rebosa o cuando no existe un control adecuado sobre su llenado.

Además, la falta de monitoreo puede dificultar el uso eficiente del recurso hídrico, ya que las personas no conocen en tiempo real el nivel disponible del tanque. Esto puede generar consumo innecesario de agua y poca capacidad de reacción ante niveles muy bajos o altos.

A partir de esta situación, se planteó el desarrollo de un sistema IoT que permite medir automáticamente el nivel del agua mediante un sensor ultrasónico conectado a un ESP32. El sistema muestra el porcentaje de llenado en una pantalla OLED y genera alertas visuales y sonoras según el nivel detectado.

Desde el enfoque ABP, el proyecto busca no solo desarrollar una maqueta tecnológica, sino también proponer una solución que ayude al monitoreo y al uso más eficiente del agua mediante herramientas de Internet de las Cosas

4. Objetivos general y alcance del proyecto

Objetivo general

Diseñar, implementar y validar un sistema IoT basado en ESP32 para monitorear el nivel de agua de un tanque, mostrar la información en tiempo real y generar alertas automáticas que ayuden al control y uso eficiente del agua.

Objetivos específicos

- Implementar un sensor ultrasónico HC-SR04 para medir el nivel del agua dentro del tanque.
- Programar el ESP32 en Micro Python para procesar la información y calcular el porcentaje de llenado del tanque.
- Mostrar los datos del nivel de agua en una pantalla OLED y activar Leds y un buzzer como alertas según el nivel detectado.
- Realizar pruebas funcionales en la plataforma Wokwi para verificar el correcto funcionamiento del sistema y de las alertas automáticas.

Alcance del proyecto

Este proyecto busca diseñar y poner en funcionamiento un sistema sencillo para monitorear el nivel de agua de un tanque utilizando un ESP32. El sistema medirá la distancia entre el sensor y el agua para calcular el porcentaje de llenado y mostrar la información en una pantalla OLED de forma clara y en tiempo real.

Además, contará con Leds y un buzzer que servirán como alertas para indicar si el nivel del agua está bajo, medio o alto. Para el desarrollo del proyecto se realizará el montaje del circuito, la programación en Micro Python y las pruebas de funcionamiento mediante simulación en la plataforma Wokwi.

El proyecto está pensado como un prototipo académico para demostrar cómo se puede aplicar el Internet de las Cosas en el monitoreo del agua y en el uso más eficiente de este recurso. No pretende funcionar como un sistema industrial ni controlar procesos complejos, sino mostrar de manera práctica cómo sensores, alertas y microcontroladores pueden trabajar juntos para resolver una necesidad cotidiana.

5. Estado del arte

Para el desarrollo de este proyecto se consultaron diferentes ejemplos y proyectos relacionados con el monitoreo del nivel de agua mediante tecnologías IoT. En la mayoría de los casos, estas soluciones buscan ayudar a las personas a conocer la cantidad de agua disponible en un tanque y evitar problemas como desperdicios por rebosamiento o la falta de control sobre el consumo de agua.

Durante la revisión se encontró que muchos proyectos utilizan sensores para medir la distancia entre el agua y la parte superior del tanque. Con esta información es posible calcular el nivel de llenado y generar alertas cuando el agua alcanza valores muy bajos o altos. Estas herramientas permiten tener un mejor control del recurso y facilitan la toma de decisiones.

También se observó que uno de los dispositivos más utilizados en este tipo de proyectos es el ESP32. Su popularidad se debe a que es económico, fácil de programar y permite conectar diferentes sensores y dispositivos electrónicos. Además, incorpora conexión Wifi, lo que facilita el desarrollo de aplicaciones relacionadas con el Internet de las Cosas. En varios proyectos revisados, el ESP32 trabaja junto con sensores ultrasónicos HC-SR04 para medir el nivel del agua y mostrar los resultados en pantallas o aplicaciones de monitoreo.

Otra característica común es el uso de alertas automáticas. Algunos sistemas activan bombas de agua cuando el nivel es muy bajo, mientras que otros utilizan luces o alarmas sonoras para avisar al usuario sobre el estado del tanque. Estas funciones ayudan a mejorar el control del agua y a reducir posibles desperdicios.

Respecto a la visualización de la información, existen proyectos que envían los datos a plataformas como ThingSpeak, Blynk o Node-RED para consultarlos desde internet. Sin embargo, para este proyecto se decidió utilizar una pantalla OLED, ya que el objetivo principal es demostrar de forma sencilla el funcionamiento del sistema y observar los resultados directamente durante la simulación.

La elección del sensor ultrasónico HC-SR04 se realizó por su facilidad de uso, buena precisión y bajo costo. De igual manera, se seleccionó el ESP32 por ser una herramienta muy utilizada en proyectos educativos y de automatización. Los LEDs y el buzzer se incorporaron para generar alertas visuales y sonoras fáciles de identificar por cualquier usuario.

Este tipo de proyectos forman parte de las aplicaciones actuales del Internet de las Cosas, donde diferentes dispositivos pueden medir variables del entorno, procesar información y generar respuestas automáticas. Además, promueven un uso más responsable del agua y muestran cómo la tecnología puede contribuir al cuidado de los recursos.

Como posible mejora futura, se plantea la integración del sistema con una plataforma en la nube para consultar la información desde un teléfono celular o computador. También podría incorporarse el control

automático de una bomba de agua para gestionar el llenado del tanque sin necesidad de intervención constante del usuario.

6. Materiales, herramientas y consideraciones de seguridad

Para el desarrollo del proyecto se utilizaron diferentes componentes electrónicos y herramientas de software que permitieron diseñar, programar y simular el sistema de monitoreo de nivel de agua. Cada elemento cumple una función específica dentro del funcionamiento del prototipo.

Hardware

Componente	Referencia	Cantidad	Función	Voltaje / Interfaz
Microcontrolador	ESP32	1	Procesar la información del sistema y controlar los dispositivos conectados.	3.3 V / Wi-Fi
Sensor ultrasónico	HC-SR04	1	Medir la distancia entre el sensor y el nivel del agua.	5 V / Digital
Pantalla OLED	SSD1306	1	Mostrar el porcentaje de llenado y el estado del tanque.	3.3 V / I2C
LED verde	LED 5 mm	1	Indicar nivel alto de agua.	3.3 V
LED amarillo	LED 5 mm	1	Indicar nivel medio de agua.	3.3 V
LED rojo	LED 5 mm	1	Indicar nivel bajo de agua.	3.3 V
Buzzer	Buzzer activo	1	Generar una alerta sonora cuando el nivel sea crítico.	3.3 V
Resistencias	330 Ω	3	Proteger los LEDs de sobrecorriente.	-
Protoboard	Protoboard estándar	1	Facilitar las conexiones del circuito.	-
Cables de conexión	Jumpers	Varios	Realizar las conexiones entre componentes.	-

Software

Herramienta	Función
Micro Python	Lenguaje utilizado para programar el ESP32.
Wokwi	Plataforma utilizada para diseñar y simular el circuito electrónico.
Thonny	Entorno de programación utilizado para escribir y cargar el código en el ESP32.
ThingSpeak	Plataforma en la nube utilizada para almacenar, visualizar y analizar los datos enviados por el ESP32 mediante gráficas en tiempo real.
GitHub	Repositorio utilizado para almacenar el código, la documentación y las evidencias del proyecto.

Herramientas adicionales

Computador con acceso a Wokwi y al entorno de programación.

Consideraciones de seguridad

Aunque la mayor parte del proyecto se realizó mediante simulación en Wokwi, es importante tener en cuenta algunas recomendaciones de seguridad en caso de construir el prototipo físicamente. Antes de encender el circuito, se deben revisar todas las conexiones para asegurarse de que los componentes estén conectados correctamente y evitar posibles daños por errores de cableado.

También es importante utilizar los voltajes adecuados para cada componente, ya que una conexión incorrecta podría afectar el funcionamiento del ESP32, el sensor ultrasónico o la pantalla OLED. Los LEDs deben conectarse con sus respectivas resistencias para evitar que se dañen por exceso de corriente.

Si el sistema se utiliza cerca de agua o en un tanque real, se recomienda mantener los componentes electrónicos protegidos y alejados de la humedad. De esta manera se reducen los riesgos de cortocircuitos y se garantiza un funcionamiento más seguro del sistema.

En general, se trata de un proyecto de bajo voltaje y bajo riesgo; sin embargo, siempre es recomendable trabajar con cuidado, revisar las conexiones antes de energizar el circuito y realizar pruebas de manera responsable.

7. Diseño electrónico, simulación y montaje

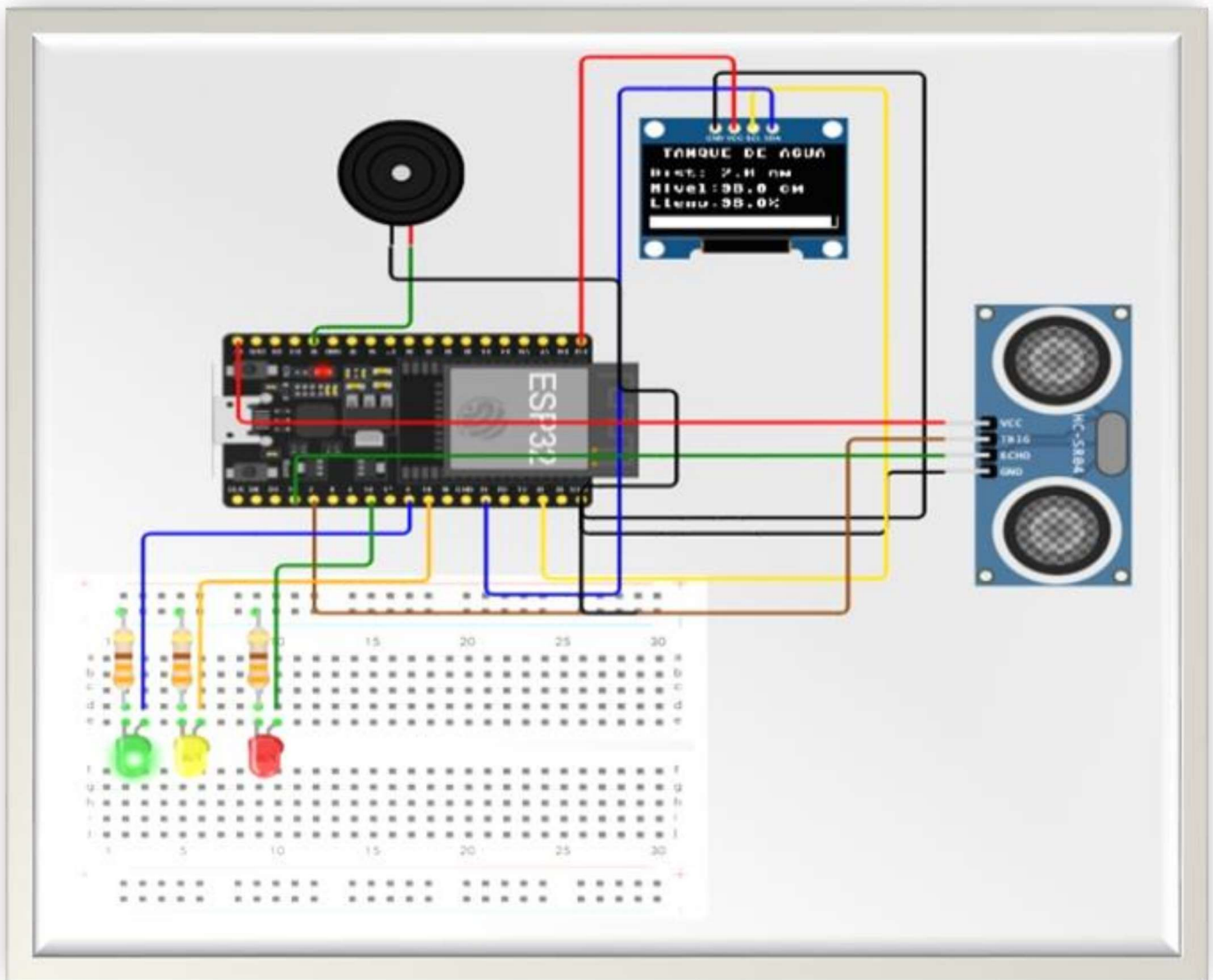


Figura 1. Esquema de montaje electrónico del sistema de monitoreo de nivel de tanque.

La figura 1 muestra el montaje electrónico realizado en la simulación del proyecto de monitoreo de nivel de agua. En el circuito se integran un ESP32, un sensor ultrasónico HC-SR04, una pantalla OLED, tres LEDs, un buzzer y una protoboard para organizar las conexiones.

El ESP32 es el encargado de controlar todo el sistema y procesar la información recibida por el sensor ultrasónico. El HC-SR04 mide la distancia entre el sensor y el nivel del agua para calcular qué tan lleno se encuentra el tanque. Esta información se muestra en tiempo real en la pantalla OLED, donde se puede observar la distancia medida, el porcentaje de llenado y el estado del tanque.

Los LEDs funcionan como indicadores visuales del nivel del agua: el verde indica nivel alto, el amarillo nivel medio y el rojo nivel bajo. Además, el buzzer actúa como una alerta sonora cuando el sistema detecta niveles críticos.

La simulación permitió comprobar que las conexiones y el funcionamiento del sistema fueran correctos antes de realizar un montaje físico. También facilitó las pruebas del código en MicroPython y la validación de las alertas y la visualización de datos en pantalla.

8. Algoritmo o código empleado

```

from machine import Pin, PWM, time_pulse_us, I2C
from time import sleep
from umqtt.simple import MQTTClient
import network, ntptime, time
import ssd1306

WIFI_SSID = "wokwi-GUEST"
WIFI_PASSWORD = ""

THINGSPEAK_MQTT_SERVER = "mqtt3.thingspeak.com"
THINGSPEAK_CHANNEL_ID = "3395443"
MQTT_CLIENT_ID = "DTM1J5gtKXICBwMMVOTgGOSo"
MQTT_USERNAME = "DTM1J5gtKXICBwMMVOTgGOSo"
MQTT_PASSWORD = "gUxuU97SPH5/mk1CAFxeFkvs"
PUBLISH_INTERVAL = 50

# =====
# CONFIGURACION OLED
# =====

i2c = I2C(0, scl=Pin(22), sda=Pin(21), freq=100000)

print("Escaneando bus I2C...")
dispositivos = i2c.scan()
print("Dispositivos encontrados:", [hex(d) for d in dispositivos])

if len(dispositivos) == 0:
    print("ERROR: No se detecta la pantalla OLED.")
    print("Revise conexiones: VCC, GND, SCL=GPI022, SDA=GPI021.")
    oled = None
else:
    direccion_oled = dispositivos[0]
    print("OLED detectada en:", hex(direccion_oled))
    oled = ssd1306.SSD1306_I2C(128, 64, i2c, addr=direccion_oled)

# =====
# CONFIGURACION HC-SR04
# =====

TRIG = Pin(2, Pin.OUT)
ECHO = Pin(15, Pin.IN)

# =====
# CONFIGURACION LEDS
# =====

led_vacio = Pin(16, Pin.OUT)
led_normal = Pin(18, Pin.OUT)
led_lleno = Pin(5, Pin.OUT)
led_off = Pin(16, Pin.OUT)
led_on = Pin(5, Pin.OUT)
led_OK = Pin(18, Pin.OUT)

sensor = pulse_us.time_pulse_us(Pin(15))

Umbral_Centimetros = [0, 100]
Umbral_Llenado = [100, 0]

```

Figura 2. Configuración inicial del sistema.

En esta parte del código se preparan todos los elementos necesarios para que el sistema funcione correctamente. Se cargan las librerías que se van a utilizar y se configuran los componentes principales, como la pantalla OLED, el sensor ultrasónico, los LEDs, el buzzer y la conexión a internet. Es el punto de partida que permite que todo el sistema trabaje de manera conjunta.


```

def clear():
    print("\033[2J\033[H")

def conectar_wifi():
    wlan = network.WLAN(network.STA_IF)
    wlan.active(True)
    wlan.connect(WIFI_SSID, WIFI_PASSWORD)
    print("Conectando al WiFi...", end="")
    while not wlan.isconnected():
        print(".", end="")
        Led_off.on()
        Led_on.off()
        sleep(0.5)
    print("\n WiFi Conectado. IP:", wlan.ifconfig()[0])
    Led_off.off()
    Led_on.on()
    print("Sincronizando Hora con Servidor NTP...")
    try:
        ntptime.settime()
        print("Hora Sincronizada con NTP")
        Led_off.off()
        Led_on.on()
    except Exception as e:
        print("No se Pudo Sincronizar la Hora:", e)
        Led_off.on()
        Led_on.off()

def conectar_mqtt():
    client = MQTTClient(
        THINGSPEAK_MQTT_SERVER,
        user = MQTT_USERNAME,
        password = MQTT_PASSWORD,
        port = 1883,
        ssl = False
    )
    client.connect()
    print("Conectado a ThingSpeak MQTT como Device.")
    Led_off.off()
    Led_on.on()
    return client

def enviar_datos(client, centimetros, llenado):
    topic = f"channels/{THINGSPEAK_CHANNEL_ID}/publish"
    payload = f"field1={centimetros}&field2={llenado}"
    client.publish(topic, payload)
    Led_off.off()
    Led_on.on()

def hora_local():
    t = time.localtime()
    UTC_Colombia = -5
    hora = (t[3] + UTC_Colombia) % 24
    minuto = t[4]
    segundo = t[5]
    return hora, minuto, segundo

conectar_wifi()
mqtt_client = conectar_mqtt

```

Figura 3. Conexión WiFi y envío de datos a ThingSpeak.

Aquí se realiza la conexión del ESP32 a la red WiFi y se configura la comunicación con ThingSpeak. Gracias a esto, los datos obtenidos por el sistema pueden enviarse a la nube para ser almacenados y consultados posteriormente mediante gráficas y registros.

```

def medir_distancia():
    TRIG.off()
    time.sleep_us(2)

    TRIG.on()
    time.sleep_us(10)
    TRIG.off()

    duracion = time_pulse_us(ECHO, 1, 30000)

    if duracion < 0:
        return None

    distancia = (duracion * 0.0343) / 2
    return distancia

def actualizar_leds_y_alarma(distancia):
    global alerta_lleno

    print("distancia en función:", distancia)

    if distancia > 75:
        led_vacio.on()
        led_normal.off()
        led_lleno.off()
        alerta_lleno = False
        print("antes de sonar")
        sonar(800, 0.15)
        print("despues de sonar")

    elif distancia > 25:
        led_vacio.off()
        led_normal.on()
        led_lleno.off()
        alerta_lleno = False

    else:
        led_vacio.off()
        led_normal.off()
        led_lleno.on()
        if not alerta_lleno:
            sonar(800, 0.3)
            alerta_lleno = True

```

Figura 4. Lectura del sensor y generación de alertas.

En este fragmento se encuentra la parte encargada de medir el nivel del agua utilizando el sensor ultrasónico. Una vez obtenida la medición, el sistema determina si el nivel es bajo, medio o alto y activa las alertas correspondientes mediante los LEDs y el buzzer para informar al usuario sobre el estado del tanque.

```

def dibujar_barra(distancia):

    x0 = 0
    y0 = 54
    ancho_total = 128
    alto_total = 9

    # Borde superior e inferior
    for x in range(x0, x0 + ancho_total):
        oled.pixel(x, y0, 1)
        oled.pixel(x, y0 + alto_total, 1)

    # Borde izquierdo y derecho
    for y in range(y0, y0 + alto_total + 1):
        oled.pixel(x0, y, 1)
        oled.pixel(x0 + ancho_total - 1, y, 1)

    # Relleno proporcional
    ancho_llenado = int((porcentaje / 100) * (ancho_total - 2))

    if ancho_llenado < 0:
        ancho_llenado = 0

    if ancho_llenado > ancho_total - 2:
        ancho_llenado = ancho_total - 2

    for x in range(1, ancho_llenado + 1):
        for y in range(y0 + 1, y0 + alto_total):
            oled.pixel(x, y, 1)

def actualizar_oled(distancia, nivel, porcentaje):
    if oled is None:
        return

    oled.fill(0)

    oled.text("TANQUE DE AGUA", 8, 0)
    oled.text("Dist: {:.1f} cm".format(distancia), 0, 16)
    oled.text("Nivel:{:.1f} cm".format(nivel), 0, 28)
    oled.text("Lleno:{:.1f}%".format(porcentaje), 0, 40)

    dibujar_barra(porcentaje)

    oled.show()

```

Figura 5. Visualización de datos en la pantalla OLED.

Esta sección permite mostrar la información del sistema de forma clara y fácil de entender. En la pantalla se presentan datos como la distancia medida, el porcentaje de llenado y el nivel del tanque, ayudando al usuario a conocer el estado del agua en tiempo real.

```

def mostrar_error_sensor():
    if oled is not None:
        oled.fill(0)
        oled.text("Error sensor", 0, 0)
        oled.text("HC-SR04", 0, 15)
        oled.show()

# =====
# PROGRAMA PRINCIPAL
# =====

if oled is not None:
    oled.fill(0)
    oled.text("Prueba local", 0, 0)
    oled.text("OLED + HC-SR04", 0, 15)
    oled.show()

time.sleep(1)

while True:
    distancia = medir_distancia()

    if distancia is None:
        print("Error de lectura del sensor.")
        mostrar_error_sensor()
        time.sleep(0.5)
        continue

    nivel = ALTURA_TANQUE - distancia

    if nivel < 0:
        nivel = 0

    if nivel > ALTURA_TANQUE:
        nivel = ALTURA_TANQUE

    porcentaje = (nivel / ALTURA_TANQUE) * 100

    print("Distancia: {:.2f} cm | Nivel: {:.2f} cm | Llenado: {:.2f}%".format(
        distancia,
        nivel,
        porcentaje
    ))

    hora, minuto, segundo = hora_local()
    print(f"Hora Actual: {hora:02d}:{minuto:02d}:{segundo:02d}")

    enviar_datos(mqtt_client, centimetros, llenado)
    sleep(PUBLISH_INTERVAL)

    actualizar_leds_y_alarma(distancia)
    actualizar_oled(distancia, nivel, porcentaje)

    time.sleep(0.5)

```

Figura 6. Funcionamiento principal del sistema.

En esta parte se encuentra la lógica principal del programa. El sistema realiza continuamente las mediciones, actualiza la información mostrada en pantalla, verifica las condiciones del tanque, activa las alertas cuando es necesario y prepara los datos para enviarlos a ThingSpeak.

```

while True:
    try:
        sensor.measure()
        centimetros = sensor.centimetros()
        llenado = sensor.llenado()

        led_OK.on()
        time.sleep(0.1)
        led_OK.off

        if centimetros < Umbral_Centimetros[0] or centimetros > Umbral_Centimetros[1]:
            led_HUM.on()
        else:
            led_HUM.off()

        if llenado < Umbral_Llenado[0] or llenado > Umbral_Llenado[1]:
            led_TEMP.on()
        else:
            led_TEMP.off()

        clear()
        print("Centimetros: ", centimetros, "°C")
        print("Llenado: ", llenado, "%")

        hora, minuto, segundo = hora_local()
        print(f"Hora Actual: {hora:02d}:{minuto:02d}:{segundo:02d}")

    except Exception as e:
        print("Error General:", e)
        Led_on.off()
        Led_off.on()
        sleep(5)

# =====
# PARAMETROS DEL TANQUE
# =====

ALTURA_TANQUE = 100 # cm
alerta_lleno = False

def sonar(frecuencia, duracion_seg):
    b = PWM(Pin(13), freq=frecuencia, duty=512)
    time.sleep(duracion_seg)
    b.deinit()

```

Figura 7. Verificación de lecturas y pruebas del sistema.

Este fragmento permite comprobar que las mediciones realizadas por el sensor sean correctas. Además, muestra información en el monitor serial para facilitar las pruebas y ayudar a identificar posibles errores durante el funcionamiento del prototipo.

9. Integración con nube, dashboard y visualización

Para este proyecto se utilizó **ThingSpeak** como plataforma para guardar y visualizar los datos enviados por el ESP32. Gracias a esta herramienta fue posible observar cómo cambiaban las mediciones del sistema durante las pruebas y verificar que la información se estaba enviando correctamente.

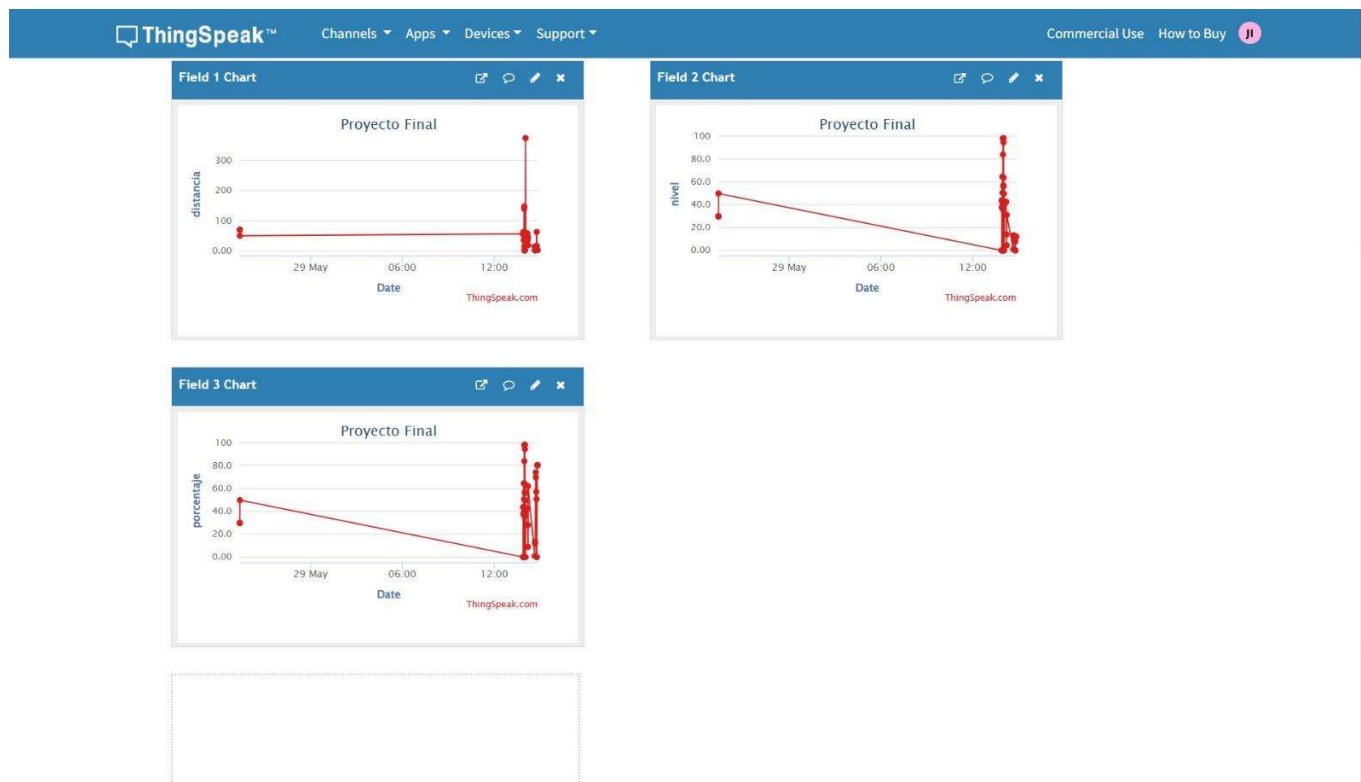


Figura 8. Dashboard de ThingSpeak utilizado para el monitoreo del nivel de agua.

La Figura 8 muestra las gráficas generadas en ThingSpeak a partir de los datos enviados por el sistema. En ellas se pueden observar tres variables importantes: la distancia medida por el sensor ultrasónico, el nivel del tanque y el porcentaje de llenado.

La primera gráfica corresponde a la distancia detectada por el sensor. Los cambios que se observan reflejan las diferentes pruebas realizadas durante la simulación, donde se modificó el nivel del agua para comprobar el funcionamiento del sistema.

La segunda gráfica muestra el nivel del tanque. Esta información permite identificar de forma rápida si el tanque se encuentra con poca agua, en un nivel intermedio o cerca de su capacidad máxima.

La tercera gráfica representa el porcentaje de llenado. Esta es una de las variables más fáciles de interpretar, ya que permite conocer de manera inmediata qué tan lleno se encuentra el tanque en cada momento.

El uso de ThingSpeak fue de gran ayuda porque permitió verificar que los datos se estaban enviando y almacenando correctamente. Además, facilitó el seguimiento de las mediciones durante las pruebas y permitió observar el comportamiento del sistema de una manera más visual y organizada.

Durante algunas pruebas se presentaron cambios bruscos en las gráficas, lo cual puede deberse a variaciones en las mediciones del sensor o a los ajustes realizados durante la simulación. Sin embargo, en general los resultados obtenidos permitieron comprobar que el sistema funcionó correctamente y que la comunicación entre el ESP32 y la plataforma se realizó de forma satisfactoria.

10. Protocolo de pruebas, resultados y análisis

Descripción de la prueba realizada

Para comprobar que el sistema funcionaba correctamente, se realizó una prueba conectando el ESP32, el sensor ultrasónico HC-SR04 y los LEDs indicadores. Durante la prueba se simularon diferentes niveles de agua para verificar que el sensor detectara los cambios de distancia y enviara la información a la plataforma ThingSpeak.

Como se observa en la imagen, los datos fueron registrados y mostrados en tiempo real mediante gráficas. Además, los LEDs respondieron de acuerdo con el nivel detectado, indicando visualmente el estado del tanque.

Los resultados obtenidos demostraron que el sistema mide correctamente el nivel del tanque, procesa la información y la transmite de forma exitosa a Internet, permitiendo consultar los datos de manera remota. Esto confirma que la solución propuesta cumple con su objetivo de monitorear el nivel del tanque de forma sencilla y confiable.

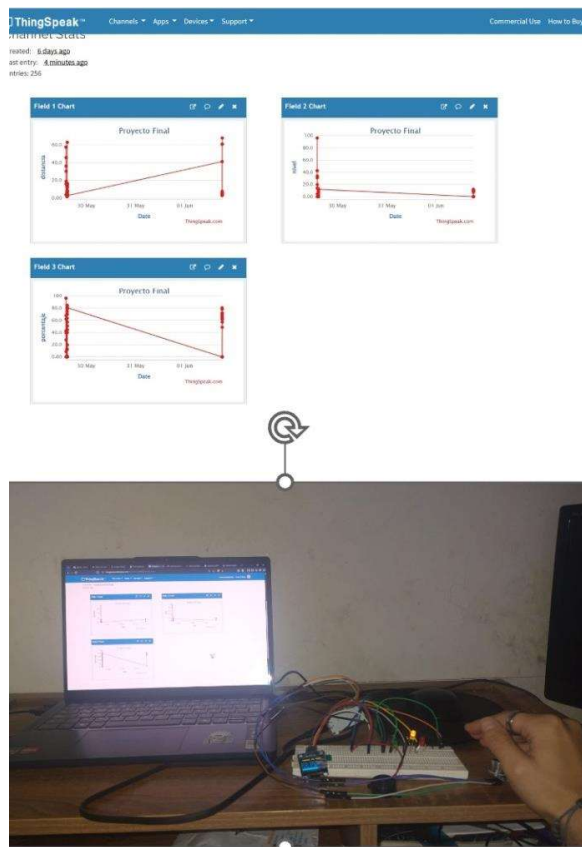


Figura 9. Prueba de funcionamiento del sistema de monitoreo de nivel de tanque con ESP32, sensor ultrasónico HC-SR04 y plataforma ThingSpeak.

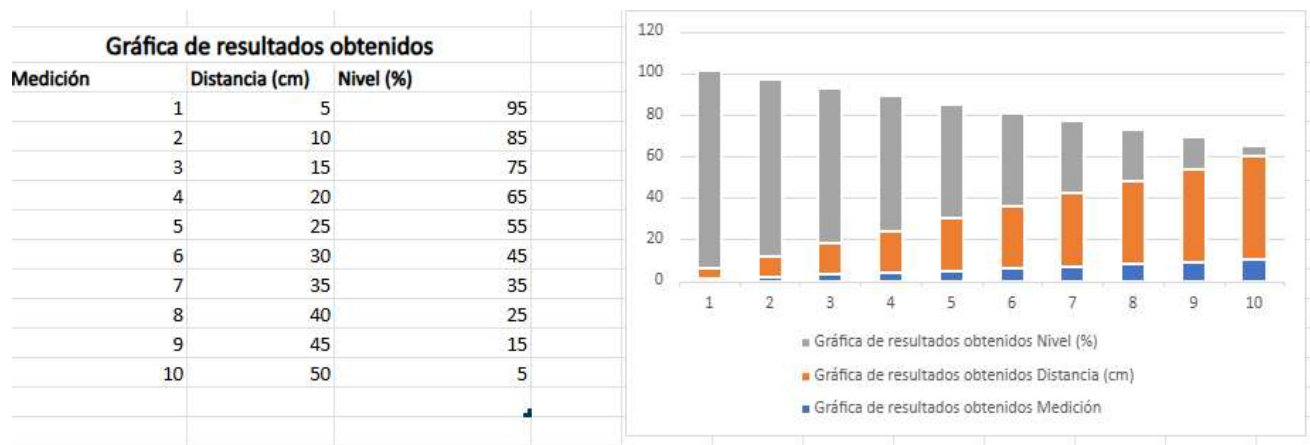


Figura 10. Gráfica de resultados obtenidos durante las pruebas del sistema de monitoreo de nivel de tanque. Se observa que, a medida que aumenta la distancia medida por el sensor ultrasónico, el porcentaje de llenado disminuye, evidenciando el correcto funcionamiento del sistema para detectar y reportar el nivel del tanque en tiempo real.

Descripción de la gráfica

La gráfica muestra la relación entre la distancia medida por el sensor y el nivel de llenado del tanque durante las diferentes pruebas realizadas. Se puede observar que, a medida que la distancia aumenta de 5 cm a 50 cm, el porcentaje de nivel disminuye de 95 % a 5 %.

Este comportamiento era el esperado, ya que cuando el nivel del líquido baja, la distancia entre el sensor y la superficie del agua aumenta. Los resultados obtenidos demuestran que el sensor ultrasónico detecta correctamente los cambios de nivel y que el sistema realiza de manera adecuada el cálculo del porcentaje de llenado.

Gracias a estas mediciones, la información pudo ser enviada y visualizada en ThingSpeak en tiempo real, confirmando el correcto funcionamiento del sistema de monitoreo del tanque. En general, la prueba permitió verificar que la solución propuesta es capaz de medir y reportar el nivel del tanque de forma confiable y precisa.

11. Video demostrativo y evidencias audiovisuales

12. Impacto ético, social, ambiental y de sostenibilidad

El desarrollo de este proyecto demuestra cómo la tecnología puede ayudar a mejorar el monitoreo y control de recursos tan importantes como el agua. Aunque en ciudades como Medellín el suministro de agua potable es constante y los tanques de almacenamiento no son tan comunes en todos los hogares, este tipo de sistemas puede ser útil en viviendas, empresas, instituciones o zonas donde se utilizan tanques de reserva.

Desde el punto de vista social, el sistema permite conocer el nivel de agua disponible de forma rápida y sencilla, evitando revisiones manuales y facilitando la toma de decisiones. Además, puede contribuir a prevenir desperdicios de agua ocasionados por rebosamientos o por un mal control del almacenamiento.

En el aspecto ambiental, el proyecto promueve un uso más responsable del recurso hídrico, ya que permite hacer seguimiento al consumo y al nivel de almacenamiento del agua. Esto puede ayudar a reducir pérdidas innecesarias y fomentar una mayor conciencia sobre el cuidado de este recurso.

Sin embargo, también existen algunas limitaciones. Si el sensor presenta errores en la medición o las conexiones no funcionan correctamente, el sistema podría mostrar información inexacta o generar alertas equivocadas. Por esta razón, es importante realizar pruebas, verificar las conexiones y revisar periódicamente el funcionamiento de los componentes.

En cuanto a la sostenibilidad, el prototipo utiliza componentes electrónicos de bajo consumo energético y puede construirse utilizando materiales reutilizables como protoboards, cables y módulos electrónicos que pueden emplearse en otros proyectos académicos. Además, se recomienda proteger las credenciales de conexión WiFi y las claves de acceso a las plataformas utilizadas para evitar el uso indebido de la información.

13. Conclusiones y aprendizajes

A partir de las pruebas realizadas, se puede concluir que el prototipo cumplió con el objetivo principal de monitorear el nivel de agua mediante un sensor ultrasónico y mostrar la información de manera clara al usuario. El sistema logró medir la distancia, calcular el porcentaje de llenado y generar alertas visuales según el estado del tanque.

También se comprobó que el ESP32 pudo enviar correctamente los datos a ThingSpeak, permitiendo almacenar y visualizar la información mediante gráficas. Esto permitió verificar la integración entre el hardware, la programación y la plataforma de monitoreo utilizada en el proyecto.

Durante el desarrollo del trabajo se fortalecieron conocimientos relacionados con programación en MicroPython, conexiones electrónicas, uso de sensores, manejo del ESP32 y herramientas de visualización de datos en la nube. Asimismo, se adquirió experiencia en la realización de pruebas, solución de errores y trabajo en equipo.

Entre las principales limitaciones encontradas se destacan las pequeñas variaciones en las mediciones del sensor ultrasónico y las posibles interrupciones de conexión a internet que pueden afectar el envío de datos. Aunque estas situaciones no impidieron el funcionamiento del sistema, muestran aspectos que podrían mejorarse en futuras versiones.

Como trabajo futuro, se podría incorporar una bomba de agua para automatizar el llenado del tanque, desarrollar una aplicación móvil para consultar la información desde cualquier lugar y mejorar la precisión de las mediciones mediante filtros o promedios de lectura.

En general, este proyecto permitió comprender de manera práctica cómo funciona una solución IoT, integrando sensores, procesamiento de datos, visualización de información y comunicación en la nube para resolver una necesidad relacionada con el monitoreo del agua.

14. Referencias y anexos

- Espressif Systems. (2024). *ESP32 Series Datasheet*. Recuperado de: <https://www.espressif.com>
- Adafruit Industries. (2024). *SSD1306 OLED Displays Documentation*. Recuperado de: <https://learn.adafruit.com>
- SparkFun Electronics. (2024). *HC-SR04 Ultrasonic Distance Sensor Hookup Guide*. Recuperado de: <https://learn.sparkfun.com>

- MathWorks. (2024). *ThingSpeak Documentation*. Recuperado de:
<https://thingspeak.com>
- Wokwi. (2024). *Wokwi ESP32 Simulator Documentation*. Recuperado de:
<https://docs.wokwi.com>
- MicroPython Foundation. (2024). *MicroPython Documentation*. Recuperado de:
<https://docs.micropython.org>
- Random Nerd Tutorials. (2024). *ESP32 Projects and Tutorials*. Recuperado de:
<https://randomnerdtutorials.com>